

How to use Command Line Interface (CLI)

Innovation Analytics Lab.

Global Master of Business Administration

Tunghai University

2026

Table of Contents

1. Purpose
2. Essential Terminology (TUI Glossary)
3. How to Open the Terminal / Command Prompt
4. Common Command Line Operations (Quick Reference)
5. Detailed Examples & Explanations
6. Student Evaluation: Real-World Scenario

1. Purpose

This tutorial introduces the **command line interface (CLI)**, a way to control your computer by typing text commands, rather than clicking icons, on macOS/Linux (Unix) and Windows. Proficiency in the CLI is a foundational skill for business and non-engineering students enrolled in artificial intelligence and machine learning courses.

```
C:\WINDOWS\system32\cmd.exe
** Visual Studio 2022 Developer Command Prompt v17.14.8
** Copyright (c) 2025 Microsoft Corporation

C:\Windows\System32>clrcver

Microsoft (R) .NET CLR Version Tool  Version 4.8.3928.0
Copyright (c) Microsoft Corporation. All rights reserved.

Versions installed on the machine:
v4.8.39319

C:\Windows\System32>
```

```
john@Johns-MacBook-Pro ~ % sudo /Applications/Install\ macOS\ Ventura.app/Contents
/Resources/createinstallmedia --volume /Volumes/MyVolume
Password:
Ready to start.
To continue we need to erase the volume at /Volumes/MyVolume.
If you wish to continue type [Y] then press return: Y
Erasing disk: 0%... 10%... 20%... 30%... 100%
Making disk bootable...
Copying to disk: 0%... 10%... 20%... 30%... 40%... 50%... 100%
Install media now available at "/Volumes/Install macOS Ventura"
john@Johns-MacBook-Pro ~ %
```

2. Essential Terminology (TUI Glossary)

Directory (Folder)	A location in a file system used to store and organize files and other directories.
Parent Directory	A directory that contains subdirectories. E.g., <code>climate_research</code> is the parent of <code>src</code> .
Child Directory	A directory inside another. E.g., <code>src</code> is a child of <code>climate_research</code> .
Current Working Dir	The directory you are currently "inside" in the terminal.
Home Directory	Default personal directory (<code>~</code> on macOS/Linux, <code>C:\Users\Username</code> on Windows).
Root Directory	Top-most directory (<code>/</code> on macOS/Linux, <code>C:\</code> on Windows).
Project Root Dir	The top-most directory specific to a project, containing all related files and subdirectories.
Absolute Path	Complete path from the root (e.g., <code>/home/user/climate_research</code>).
Relative Path	Path relative to current dir (e.g., <code>src/main.py</code> or <code>../references</code>).
Terminal / Shell	The text-based application (TUI) used to type and execute commands.

3. How to Open the Terminal / Command Prompt

macOS Terminal

- **Spotlight:** Cmd + Space → type Terminal → Enter
- **Applications:** Finder → Applications → Utilities → Terminal

Windows Command Prompt

- **Start Menu:** Win key → type cmd → Enter
- **Run Dialog:** Win + R → type cmd → Enter

Linux Terminal

- **Keyboard Shortcut:** Ctrl + Alt + T (Ubuntu & most distros)
- **Application Launcher:** Search for Terminal or Console in app menu

4. Common Commands: Navigation

Action	macOS / Linux	Windows (Cmd)
List files	ls (or ls -la)	dir
Print working dir	pwd	cd
Move to directory	cd directory_name	cd directory_name
Move to parent	cd ..	cd ..

4. Common Commands: Managing Directories

Action	macOS / Linux	Windows (Cmd)
Create directory	<code>mkdir directory_name</code>	<code>mkdir directory_name</code>
Create subdirectory	<code>mkdir -p parent/sub</code>	<code>mkdir parent\sub</code>
Copy directory	<code>cp -r src dest</code>	<code>xcopy /E /I src dest</code>
Delete directory	<code>rm -rf directory_name</code>	<code>rmdir /s directory_name</code>

4. Common Commands: Managing Files

Action	macOS / Linux	Windows (Cmd)
Create text file	<code>touch file.txt</code>	<code>type nul > file.txt</code>
Open in GUI editor	<code>touch f && open -e f</code>	<code>notepad file.txt</code>
View file contents	<code>cat file.txt</code>	<code>type file.txt</code>
Rename	<code>mv old_name new_name</code>	<code>ren old_name new_name</code>
Delete file	<code>rm file.txt</code>	<code>del file.txt</code>

4. Common Commands: Directory Structure

Action	macOS / Linux	Windows (Cmd)
Display tree	<code>tree</code>	<code>tree /F</code>

Tip: On macOS/Linux, install via `brew install tree` or use `find ..`. On Windows, `/F` shows files; without it, only folders are listed.

5. Detailed Examples: Setup & Navigation

Setup: Create a practice directory and navigate into it:

```
mkdir terminal_practice  
cd terminal_practice
```

List Files & Directories

```
# macOS/Linux  
ls  
ls -la  
  
# Windows  
dir
```

Interpreting output: d = directory, - = file, l = symlink. On Windows: <DIR> = directory, number = file.

Print Working Dir & Navigate

```
# Print current location  
pwd          # macOS/Linux  
cd           # Windows  
  
# Move into a directory  
mkdir documentation  
cd documentation  
  
# Move to parent directory  
cd ..
```

5. Detailed Examples: Managing Directories

Create a directory:

```
mkdir src
```

Verify with `ls` or `dir`.

Create nested subdirectories:

```
# macOS/Linux
mkdir -p paper/drafts

# Windows
mkdir paper\drafts
```

Copy a directory:

```
# macOS/Linux
cp -r src src_backup

# Windows
xcopy /E /I src src_backup
```

Rename a directory (documentation → docs):

```
# macOS/Linux
mv documentation docs

# Windows
ren documentation docs
```

5. Detailed Examples: Managing Files

Create Empty File

```
# macOS/Linux
touch notes.txt

# Windows
type nul > notes.txt
```

Open in GUI Editor

```
# macOS
touch notes.txt && open -e notes.txt

# Windows
notepad notes.txt
```

View File Contents

```
# macOS/Linux
cat notes.txt

# Windows
type notes.txt
```

Delete a File

```
# macOS/Linux
rm notes.txt

# Windows
del notes.txt
```

Visualizing the Directory Structure

```
# macOS/Linux
tree

# Windows
tree /F
```

On macOS, install via `brew install tree` or use `find ..`. On Windows, `/F` shows files.

6. Student Evaluation: Scenario

Setting up a Research Project Workspace: The practice root directory and all subdirectories must include your English name (e.g., james):

Type	Example Name
Project root directory	climate_research_james
Subdirectories	src_james, output_james, paper_james/drafts_james, references_james, doc_james
File in doc	todo.txt: first line is your name, then task list

6. Step-by-Step Practical Assessment

Step 1: Open the Terminal / Command Prompt.

Step 2: Initialize Create `climate_research_[yourname]`, move into it.

Step 3: Structure Create `src_`, `output_`, `paper_/drafts_`, `references_`, `doc_subdirectories`.

Step 4: Files Create `doc_[name]/todo.txt` (name + task list), `references_[name]/sources.txt`.

Step 5: Rename `doc_[name]` → `documentation_[name]`, `sources.txt` → `citations.txt`.

Step 6: Cleanup Navigate to root, delete `output_[name]`.

Step 7: Verify While inside `climate_research_[yourname]`, run:

```
# macOS/Linux
date && tree && cat documentation_[yourname]/todo.txt

# Windows (Cmd)
echo %date% %time% && tree /F && type documentation_[yourname]\todo.txt
```

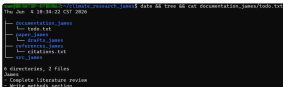
Step 8: Exit Navigate back to the parent directory with `cd ..`

6.3 Deliverables

Directory Structure Screenshot

Take a screenshot of the entire terminal window showing:

- Execution of the verification command (`date && tree && cat documentation_james/todo.txt` or Windows equivalent)
- Your custom root directory name `climate_research_[yourname]`
- All named subdirectories
- The printed timestamp



```

james@iMac: ~/Climate_research_james$ date && tree && cat documentation_james/todo.txt
Thu Jan  4 18:34:22 CST 2020
.
├── documentation_james
│   ├── todo.txt
│   ├── paper_james
│   │   └── drafts_james
│   ├── references_james
│   ├── citations.txt
│   └── src_james
└── 6 directories, 2 files
James
- Complete literature review
- Write methods section

```

Submit the screenshot to **iLearn** for grading.

6.4 Evaluation Criteria

- ❑ **Terminal Launch** : Student can open the terminal or command prompt correctly.
- ❑ **Navigation** : Student can move between nested directories without getting lost (`cd` , `cd ..`).
- ❑ **Pathing** : Student understands how to create nested directories with custom `mkdir` or Windows naming (e.g., `-p` equivalent).
- ❑ **File Management** : Student is able to create files inside specific directories and rename directories/files correctly.
- ❑ **Safety** : Student can safely delete files and empty or non-empty directories.
- ❑ **Verification** : Student is able to filter (grep) their terminal history to generate the verification report.